

Task 2

Розгортання і робота з distributed in-memory data structures на основі Hazelcast: Distributed Map

Ihor Ivanyshyn

GitHub:

https://github.com/xsponenta/software-architecture-Microservices_Basics/tree/hazelcast

- 1) Встановити і налаштувати Hazelcast:
- 2) Сконфігурувати і запустити 3 ноди (інстанси) об'єднані в кластер або як частину Java-застосування, або як окремі застосування:

Спочатку я знайшов файл config/hazelcast.xml і змінив назву кластера:

```
opt > homebrew > etc > hazelcast > hazelcast.xml
18 <!--
21 This file differs from hazelcast-default.xml in some settings
22 and it is used by the scripts in the ZIP/TGZ distribution.
23
24 This configuration, compared to hazelcast-default.xml, enables more features.
25 It's focused on keeping balance between security and usability in the distribution.
26
27 To learn how to configure Hazelcast, please see the schema at
28 http://www.hazelcast.com/schema/config/hazelcast-config-5.6.xsd
29 or the Reference Manual at https://docs.hazelcast.com/
30 -->
31
32 <!--suppress XmlDefaultAttributeValue -->
33 <hazelcast xmlns="http://www.hazelcast.com/schema/config"
34           xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
35           xsi:schemaLocation="http://www.hazelcast.com/schema/config
36                               http://www.hazelcast.com/schema/config/hazelcast-config-5.6.xsd">
37
38   <!--
39     The name of the cluster. All members of a single cluster must have the same cluster name
40     configured and a client connecting to this cluster must use it as well.
41   -->
42   <cluster-name>helloworld</cluster-name>
43
44   <properties>
45     <!--
46       ZIP and TAR distributions binds only to loopback network interface (127.0.0.1),
47       Docker image listens on all interfaces (wildcard address).
48     -->
49     <property name="hazelcast.socket.bind.any">false</property>
50   </properties>
51
52   <network>
53     <!--
54       The preferred port number where the Hazelcast instance will listen. The convention is
55       to use 5701 and it is the default both here and in various tools connecting to
56       Hazelcast. This configuration has the following attributes:
57       - port-count:
58         The default value is 100, meaning that Hazelcast will try to bind 100 ports.
59       If you set the value of port as 5701, as members join the cluster, Hazelcast tries
```

І запустив 3 ноди поступово:

```
2026-02-09 10:49:03,742 [ INFO] [main] [c.h.i.c.ClusterService]: [127.0.0.1]:5701 [helloworld] [5.6.0]
Members {size:1, ver:1} [
  Member [127.0.0.1]:5701 - 80a749c4-21f8-4788-af71-b957400e7dda this
]
```

```
2026-02-09 10:49:45,895 [ INFO] [hz.lucid_rhodes.generic-operation.thread-0] [c.h.i.c.ClusterService]: [127.0.0.1]:5702 [helloworld] [5.6.0]
Members {size:2, ver:2} [
  Member [127.0.0.1]:5701 - 80a749c4-21f8-4788-af71-b957400e7dda
  Member [127.0.0.1]:5702 - 3ca085d9-7ccb-43d7-acb9-06fe23da519e this
]
```

```
2026-02-09 10:50:36,243 [ INFO] [hz.gracious_mcclintock.generic-operation.thread-2] [c.h.i.c.ClusterService]: [127.0.0.1]:5703 [helloworld] [5.6.0]
Members {size:3, ver:3} [
  Member [127.0.0.1]:5701 - 80a749c4-21f8-4788-af71-b957400a7dda
  Member [127.0.0.1]:5702 - 3ca085d9-7ccb-43d7-acb9-06fe23da519e
  Member [127.0.0.1]:5703 - c439641c-5b5a-418b-82a9-994a5f5cb4bb this
]
```

3)Продемонструйте роботу Distributed Map:

- використовуючи API на будь-якій мові яка має клієнт для Hazelcast, створіть Distributed Map

- запишіть в неї 1000 значень з ключами від 0 до 1000

Я використав python, filename: hazelcast_task_3.py

- за допомогою Management Center подивитись на розподіл ключів по нодах

Розподіл майже рівномірний

Map Statistics (In-Memory Format: BINARY) RESET TIME 1 minute ago → now

Default View 🗑️ 📄 +

Member ^	Entries ^	Gets ^	Puts ^	Removals ^	Sets ^	Entry M
127.0.0.1:5701	337	0	337	0	0	43.
127.0.0.1:5702	333	0	333	0	0	43.
127.0.0.1:5703	330	0	330	0	0	42.
TOTAL	1 000	0	1 000	0	0	129.

1 - 3 of 3 Rows 10 ▼

- подивитись як зміниться розподіл даних по нодах:

- якщо відключити одну ноду (результати мають бути у протоколі)

```
2026-02-09 11:58:38,449 [ INFO] [hz.gracious_mcclintock.priority-generic-operation.thread-0] [c.h.i.c.ClusterService]: [127.0.0.1]:5703 [helloworld] [5.6.0]
Members {size:2, ver:4} [
  Member [127.0.0.1]:5702 - 3ca085d9-7ccb-43d7-acb9-06fe23da519e
  Member [127.0.0.1]:5703 - c439641c-5b5a-418b-82a9-994a5f5cb4bb this
]
```

Map Statistics (In-Memory Format: BINARY) RESET TIME 1 minute ago → now

Default View 🗑️ 📄 +

Member ^	Entries ^	Gets ^	Puts ^	Removals ^	Sets ^	Entry M
127.0.0.1:5702	514	0	333	0	0	66.
127.0.0.1:5703	486	0	330	0	0	63.
TOTAL	1 000	0	663	0	0	129.

1 - 2 of 2 Rows 10 ▼

Втрат даних немає

- відключити послідовно дві ноди (результати мають бути у протоколі)

```
2026-02-09 12:00:56,734 [ INFO] [hz.lucid_rhodes.cached.thread-2] [c.h.i.c.ClusterService]: [127.0.0.1]:5702 [helloworld] [5.6.0]
Members {size:1, ver:5} [
  Member [127.0.0.1]:5702 - 3ca085d9-7ccb-43d7-acb9-06fe23da519e this
]
```

RESET TIME

1 minute ago

→ now

Default View

🗑️

📄

+

Member ^	Entries ^	Gets ^	Puts ^	Removals ^	Sets ^	Entry M
127.0.0.1:5702	1 000	0	333	0	0	129.
TOTAL	1 000	0	333	0	0	129.

1 - 1 of 1 Rows

10

Перерозподілилось без втрати даних.

- відключити одночасно дві ноди (емулюючи “падіння” серверів, чи використовуючи команду **kill -9**) (результати мають бути у протоколі)

RESET TIME

1 minute ago

→ now

Default View

🗑️

📄

+

Member ^	Entries ^	Gets ^	Puts ^	Removals ^	Sets ^	Entry M
127.0.0.1:5701	317	0	317	0	0	41.
127.0.0.1:5702	329	0	329	0	0	42.
127.0.0.1:5703	354	0	354	0	0	45.
TOTAL	1 000	0	1 000	0	0	129.

1 - 3 of 3 Rows

10

Зробив той самий процес, запустив 3 ноди і записав 1000 ключів, знову розподіл близький до рівномірного

```
ihorivanyshyn@MacBook-Air-Igor ~ % ps aux | grep hazelcast
ihorivanyshyn  71773  0.7  0.1 435801088 15984 s011 S+  12:29PM  0:02.28 /Library/Frameworks/Python.framework/Versions/3.13/Resources/Python.app/Contents/MacOS/Python /Users/ihorivanyshyn/Documents/software-architecture-Microservices-Basics/hazelcast_task_3.py
ihorivanyshyn  71480  0.2  0.5 441417520 87792 s002 S+  12:27PM  0:12.59 /opt/homebrew/opt/openjdk@21/libexec/openjdk.jdk/Contents/Home/bin/java -cp /opt/homebrew/Cellar/hazelcast@5.6.0/5.6.0/libexec/lib:/opt/homebrew/Cellar/hazelcast@5.6.0/5.6.0/libexec/bin/user-lib:/opt/homebrew/Cellar/hazelcast@5.6.0/5.6.0/libexec/bin/user-lib/* --add-modules java.se --add-exports java.base/jdk.internal.ref=ALL-UNNAMED --add-opens java.base/java.lang=ALL-UNNAMED --add-opens java.base/sun.nio.ch=ALL-UNNAMED --add-opens java.management/sun.management=ALL-UNNAMED --add-opens jdk.management/com.sun.management.internal=ALL-UNNAMED -Dlog4j.shutdownHookEnabled=false -Dhazelcast.logging.shutdown=true -Dhazelcast.logging.type=log4j2 -Dlog4j.configurationFile=file:/opt/homebrew/Cellar/hazelcast@5.6.0/5.6.0/libexec/config/log4j2.properties -Dhazelcast.config=/opt/homebrew/Cellar/hazelcast@5.6.0/5.6.0/libexec/config/hazelcast.xml com.hazelcast.commandline.HazelcastServerCommandLine start
ihorivanyshyn  71587  0.2  0.6 441427136 93184 s003 S+  12:28PM  0:12.31 /opt/homebrew/opt/openjdk@21/libexec/openjdk.jdk/Contents/Home/bin/java -cp /opt/homebrew/Cellar/hazelcast@5.6.0/5.6.0/libexec/lib:/opt/homebrew/Cellar/hazelcast@5.6.0/5.6.0/libexec/lib/*:/opt/homebrew/Cellar/hazelcast@5.6.0/5.6.0/libexec/bin/user-lib:/opt/homebrew/Cellar/hazelcast@5.6.0/5.6.0/libexec/bin/user-lib/* --add-modules java.se --add-exports java.base/jdk.internal.ref=ALL-UNNAMED --add-opens java.base/java.lang=ALL-UNNAMED --add-opens java.base/sun.nio.ch=ALL-UNNAMED --add-opens java.management/sun.management=ALL-UNNAMED --add-opens jdk.management/com.sun.management.internal=ALL-UNNAMED -Dlog4j.shutdownHookEnabled=false -Dhazelcast.logging.shutdown=true -Dhazelcast.logging.type=log4j2 -Dlog4j.configurationFile=file:/opt/homebrew/Cellar/hazelcast@5.6.0/5.6.0/libexec/config/log4j2.properties -Dhazelcast.config=/opt/homebrew/Cellar/hazelcast@5.6.0/5.6.0/libexec/config/hazelcast.xml com.hazelcast.commandline.HazelcastServerCommandLine start
ihorivanyshyn  71429  0.1  0.6 441423296 186272 s000 S+  12:27PM  0:13.86 /opt/homebrew/opt/openjdk@21/libexec/openjdk.jdk/Contents/Home/bin/java -cp /opt/homebrew/Cellar/hazelcast@5.6.0/5.6.0/libexec/lib:/opt/homebrew/Cellar/hazelcast@5.6.0/5.6.0/libexec/lib/*:/opt/homebrew/Cellar/hazelcast@5.6.0/5.6.0/libexec/bin/user-lib:/opt/homebrew/Cellar/hazelcast@5.6.0/5.6.0/libexec/bin/user-lib/* --add-modules java.se --add-exports java.base/jdk.internal.ref=ALL-UNNAMED --add-opens java.base/java.lang=ALL-UNNAMED --add-opens java.base/sun.nio.ch=ALL-UNNAMED --add-opens java.management/sun.management=ALL-UNNAMED --add-opens jdk.management/com.sun.management.internal=ALL-UNNAMED -Dlog4j.shutdownHookEnabled=false -Dhazelcast.logging.shutdown=true -Dhazelcast.logging.type=log4j2 -Dlog4j.configurationFile=file:/opt/homebrew/Cellar/hazelcast@5.6.0/5.6.0/libexec/config/log4j2.properties -Dhazelcast.config=/opt/homebrew/Cellar/hazelcast@5.6.0/5.6.0/libexec/config/hazelcast.xml com.hazelcast.commandline.HazelcastServerCommandLine start
ihorivanyshyn  71835  0.0  0.0 418060096 192 s008 R+  12:33PM  0:00.00 grep hazelcast
ihorivanyshyn  71534  0.0  0.8 441550560 132328 s007 S+  12:28PM  0:31.11 /opt/homebrew/opt/openjdk@21/libexec/openjdk.jdk/Contents/Home/bin/java -server -Dloader.path=/opt/homebrew/Cellar/hazelcast-management-center/5.9.0/libexec/bin/user-lib --add-opens java.base/java.lang=ALL-UNNAMED --add-opens java.base/sun.nio.ch=ALL-UNNAMED -cp /opt/homebrew/Cellar/hazelcast-management-center/5.9.0/libexec/bin/./hazelcast-management-center-5.9.0.jar org.springframework.boot.loader.launch.PropertiesLauncher
ihorivanyshyn@MacBook-Air-Igor ~ % kill -9 71480 71587
ihorivanyshyn@MacBook-Air-Igor ~ %
```

Знаходимо всі процеси hazelcast та їх PID , kill -9 PID

Map Statistics (In-Memory Format: BINARY)

RESET TIME 1 minute ago → now

Default View

Member ^	Entries ^	Gets ^	Puts ^	Removals ^	Sets ^	Entry M
127.0.0.1:5701	662	0	317	0	0	85.
TOTAL	662	0	317	0	0	85.

1 - 1 of 1 Rows 10

- Чи буде втрата даних?

Як бачимо відбулась втрата даних.

- Яким чином зробити щоб не було втрати даних?

backup-count: Кожне значення записується на декілька нод одночасно

partition Group: дозволяє знати топологію мережі та гарантувати, що копії даних завжди знаходяться на різних фізичних машинах або в різних зонах доступності

persistence: зберігати копії даних на дисках і підвантажувати з них

4) Продемонструйте роботу Distributed Map without locks

- використовуючи 3 клієнта, на кожному з них одночасно запусить інкремент значення для одного й того самого ключа в циклі на 10K ітерацій:

```
map.putIfAbsent("key", 0);
for ( int k = 0; k < 10_000; k++ ) {
    var value = map.get( "key" );
    value++;
    map.put( "key", value );
}
```

- подивиться яке кінцеве значення для ключа "key" буде отримано (чи вийде 30K?)

Код для цього завдання у файлі hazelcast_task_4.py, та використав threading

```
import hazelcast
import threading
import time

def increment_without_lock(client):
    distributed_map = client.get_map("my-distributed-map").blocking()
    distributed_map.put_if_absent("key", 0)
    for _ in range(10000):
        value = distributed_map.get("key")
        distributed_map.put("key", value + 1)
```

```

def main():
    cluster_name = "helloworld"
    client1 = hazelcast.HazelcastClient(cluster_name=cluster_name)
    client2 = hazelcast.HazelcastClient(cluster_name=cluster_name)
    client3 = hazelcast.HazelcastClient(cluster_name=cluster_name)

    threads = [
        threading.Thread(target=increment_without_lock, args=(client1,)),
        threading.Thread(target=increment_without_lock, args=(client2,)),
        threading.Thread(target=increment_without_lock, args=(client3,))
    ]

    print("start time")
    start_time = time.time()

    for t in threads:
        t.start()

    for t in threads:
        t.join()

    end_time = time.time()

    final_value = client1.get_map("my-distributed-map").blocking().get("key")

    print(f"end without blocking = {final_value}")
    print(f"Execution time: {end_time - start_time:.2f} sec")

    client1.shutdown()
    client2.shutdown()
    client3.shutdown()

```

Результат:

```

start time
end without blocking = 13266
Execution time: 4.23 sec

```

5) Зробіть те саме з використанням песимістичним блокування та поміряйте час:

В файлі додав наступну функцію:

```
def increment_with_pessimistic_lock(client):
    distributed_map = client.get_map("my-distributed-map").blocking()
    distributed_map.put_if_absent("key", 0)
    for _ in range(10000):
        distributed_map.lock("key")
        try:
            value = distributed_map.get("key")
            distributed_map.put("key", value + 1)
        finally:
            distributed_map.unlock("key")
```

Результат:

```
start time
end with pessimistic locking = 30000
Execution time: 8.92 sec
```

6) Зробіть те саме з використанням оптимістичним блокуванням та поміряйте час:
Додав наступну функцію:

```
def implement_with_optimistic_lock(client):
    distributed_map = client.get_map("my-distributed-map").blocking()
    distributed_map.put_if_absent("key", 0)
    for _ in range(10000):
        while True:
            value = distributed_map.get("key")
            if distributed_map.replace_if_same("key", value, value + 1):
                break
```

Результат:

```
start time
end with optimistic locking = 30000
Execution time: 8.25 sec
```

7) Порівняйте результати кожного з запусків

- для реалізації без блокувань маєте спостерігати втрату даних; для реалізації з песимістичним та оптимістичним блокуванням мають бути однакові результати

- песимістичний чи оптимістичний підхід працює швидше?

Method	End value	Time(sec)
Without locking	13 266	4.23
Pessimistic lock	30 000	8.92
Optimistic lock	30 000	8.25

- Як і очікувалось в завданні є втрата даних без блокування, причиною такого є race condition
- реалізації з песимістичним та оптимістичним блокуванням дають однакові результати
- оптимістичний підхід працює швидше

8) Робота з Bounded queue

- на основі Distributed Queue (<https://docs.hazelcast.com/hazelcast/5.3/data-structures/queue#creating-an-example-queue>) налаштуйте Bounded queue на 10 елементів (<https://docs.hazelcast.com/hazelcast/5.3/data-structures/queue#configuring-queue>)
- запустіть одного клієнта який буде писати в чергу значення 1..100, а двох інших які будуть читати з черги
 - під час вичитування, кожне повідомлення має вичитуватись одразу
- яким чином будуть вичитуватись значення з черги двома клієнтами?
- перевірте яка буде поведінка на запис якщо відсутнє читання, і черга заповнена

```

opt > homebrew > etc > hazelcast > hazelcast.xml
2 <!--
14 ~ See the License for the specific language governing permissions and
15 ~ limitations under the License.
16 -->
17
18 <!--
19 The default Hazelcast ZIP/TGZ configuration.
20
21 This file differs from hazelcast-default.xml in some settings
22 and it is used by the scripts in the ZIP/TGZ distribution.
23
24 This configuration, compared to hazelcast-default.xml, enables more features.
25 It's focused on keeping balance between security and usability in the distribution.
26
27 To learn how to configure Hazelcast, please see the schema at
28 http://www.hazelcast.com/schema/config/hazelcast-config-5.6.xsd
29 or the Reference Manual at https://docs.hazelcast.com/
30 -->
31
32 <!--suppress XmlDefaultAttributeValue -->
33 <hazelcast xmlns="http://www.hazelcast.com/schema/config"
34           xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
35           xsi:schemaLocation="http://www.hazelcast.com/schema/config
36                               http://www.hazelcast.com/schema/config/hazelcast-config-5.6.xsd">
37
38   <!--
39       The name of the cluster. All members of a single cluster must have the same cluster name
40       configured and a client connecting to this cluster must use it as well.
41   -->
42   <cluster-name>helloworld</cluster-name>
43
44   <queue name="boundedQueue">
45     <max-size>10</max-size>
46   </queue>
47
48   <properties>
49     <!--
50       ZIP and TAR distributions binds only to loopback network interface (127.0.0.1),
51       Docker image listens on all interfaces (wildcard address).
52     -->
53     <property name="hazelcast.socket.bind.any">false</property>
54   </properties>
55
56   <network>
57     <!--

```

Так само як і в першому завданні в config/hazelcast.xml додаємо bounded queue

```

Produced: 1
Consumer 2 consumed: 1
Produced: 2
Consumer 1 consumed: 2
Produced: 3
Consumer 2 consumed: 3
Produced: 4
Consumer 1 consumed: 4
Consumer 2 consumed: 5
Produced: 5
Produced: 6
Consumer 1 consumed: 6
Produced: 7
Consumer 2 consumed: 7
Produced: 8
Consumer 1 consumed: 8
Produced: 9
Consumer 2 consumed: 9
Produced: 10

```



```
Produced: 93
Consumer 2 consumed: 93
Produced: 94
Consumer 1 consumed: 94
Produced: 95
Consumer 2 consumed: 95
Produced: 96
Consumer 1 consumed: 96
Produced: 97
Consumer 2 consumed: 97
Consumer 1 consumed: 98
Produced: 98
Produced: 99
Consumer 2 consumed: 99
Produced: 100
Consumer 1 consumed: 100
All processes finished.
```

Consumers успішно обробили всі записані дані.

Якщо немає Consumers:

```
Produced: 1
Produced: 2
Produced: 3
Produced: 4
Produced: 5
Produced: 6
Produced: 7
Produced: 8
Produced: 9
Produced: 10
█
```

Черга заповнюється і процес заблокований.

Висновок:

- При одночасному запуску одного продюсера та двох консюмерів дані розподіляються між читачами за принципом конкуренції.
- Кожне повідомлення з черги вичитується лише одним із консюмерів, що дозволяє паралельно обробляти великі потоки даних.
- Усі 100 значень, записані продюсером, успішно обробляються клієнтами без дублювання або втрат.
- Оскільки черга обмежена параметром 10, якщо немає Consumers після запису перших 10 елементів черга стає повною. При спробі записати 11-й елемент за умови відсутності вільних місць, метод запису блокується. Процес продюсера залишається в стані очікування до тих пір, поки один із консюмерів не вичитає елемент, звільнивши місце в черзі.